

Advanced Data Visualization

INS-605: Data Analysis



Content

1 Intro & Motivation

2 Overall view

3 Customer view

4 Product view

5 Trending view

6 Final Dashboard

1 Intro & Motivation

1.1 Introduction

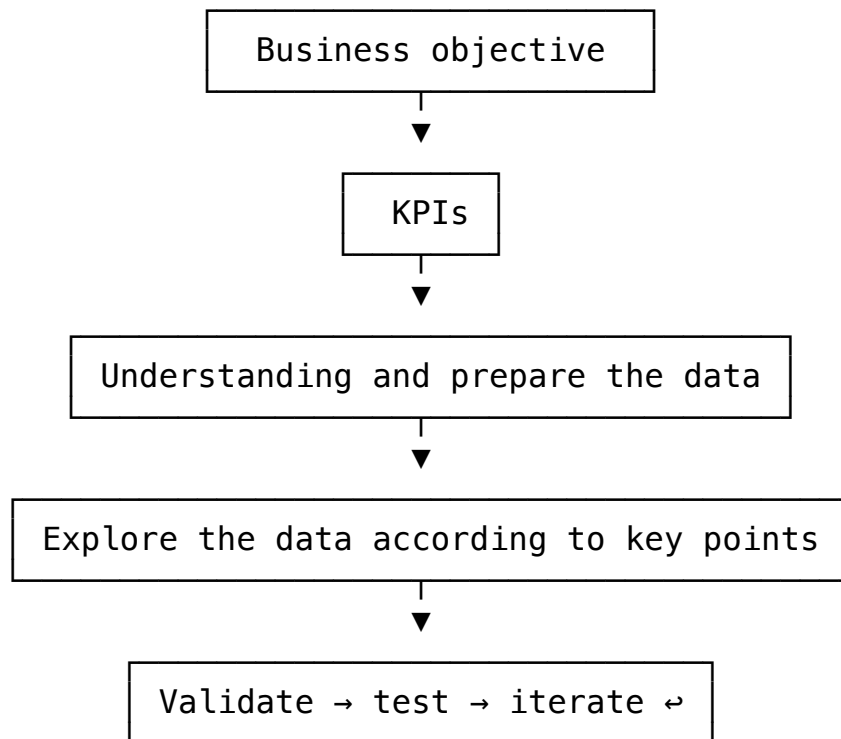
- In *Data Visualization* of **Data Analysis I**, we learned:
 - How to inspect a single variable individually
 - Uncover relationship between multiple variables
 - Detect trend / evolution of variables...
- All these are to extract knowledge from the data only.
- We now learn **additional tools** to build a presentable **Dashboard**.



- We will use **Amazon Product Rating** as a case study.

1.2 Dashboarding Process

- General view of dashboarding:

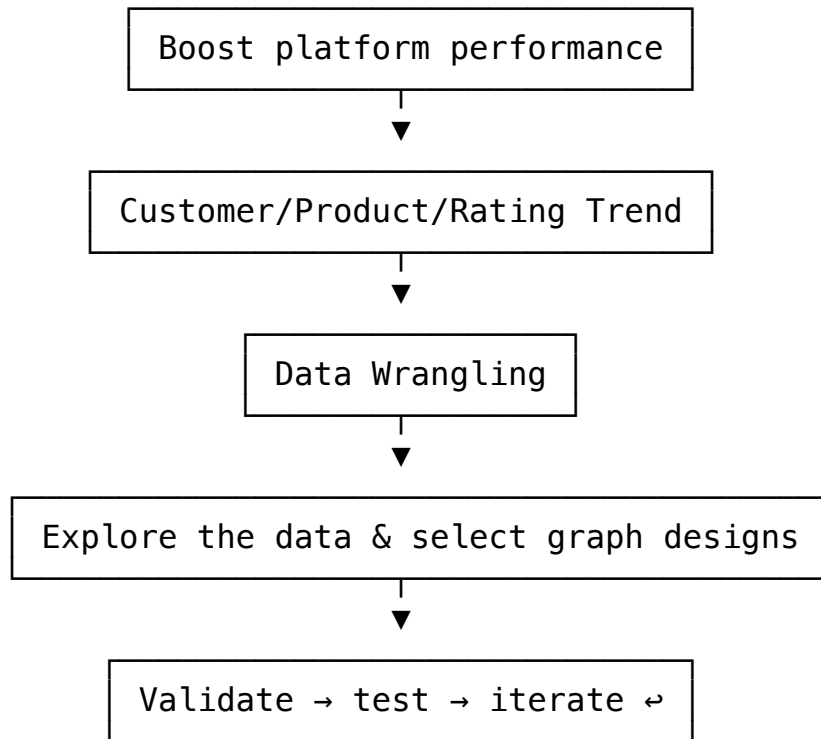


- **Key:** Business objective drives everything a dashboard will cover.

- **Business objective:** Why does this dashboard exist? Audiences?
- **Key Point Indicators:** Measurable points supporting the objective. Ex:
 - Number of Products
 - Number of Ratings
 - Average Rating
 - % of High Ratings...
- **Data wrangling:** Prepare the data.
- **Explore the data:** Analyze toward the KPIs & select visual designs.
- **Validate → test/feedback → revise ↺.**

1.2 Dashboarding Process (Cont.)

- Ex: [Amazon Product Rating](#).

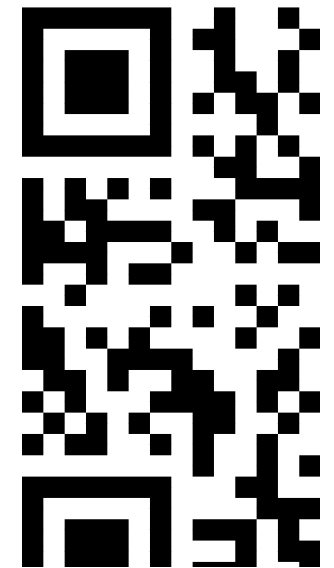


- **Target audience:** Marketing, sales and product managers.

- **Q1:** What 3-4 KPIs would you highlight on the dashboard?

slido

Join at
slido
.com
#2969

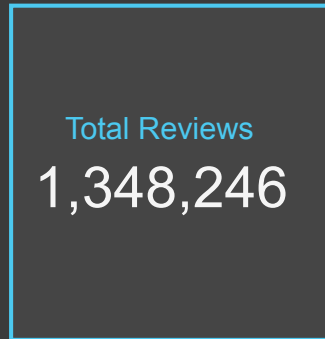


2 Overall View

2.1 Overview Dashboard

Amazon Product Reviews — Overall Market View

Total Reviews



Total Customers



Total Products



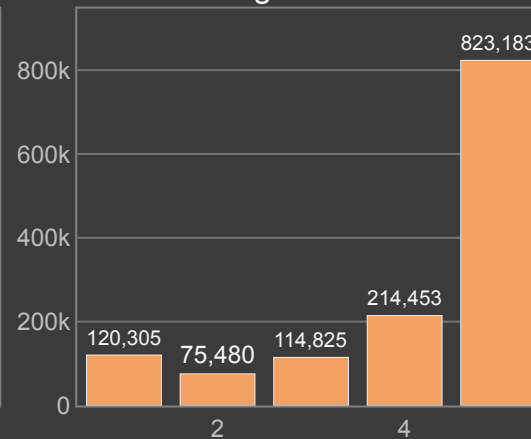
Average Rating



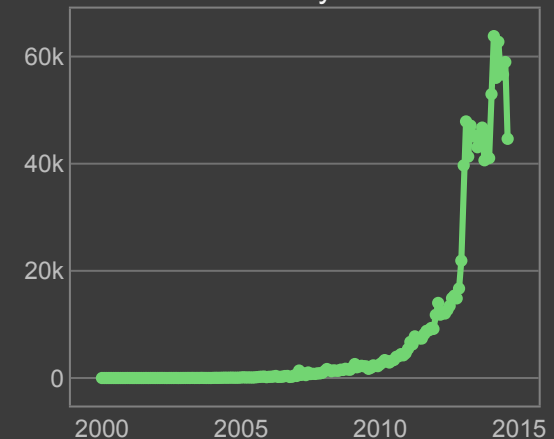
Reviews by Product Type



Rating Distribution



Review Activity Over Time



Data Wrangling

- Quick column normalization:

```
1 # Normalize cols
2 data.columns = (
3     data.columns
4     .str.strip()
5     .str.lower()
6 )
7 # Map new names:
8 rename_map = {}
9 possible_names = {
10     "userid": "user_id",
11     "productid": "product_id",
12     "producttype": "product_type",
13     "rating": "rating",
14     "timestamp": "timestamp",
15 }
16 for col in data.columns:
17     if col in possible_names:
18         rename_map[col] = possible_names[col]
19 data = data.rename(columns=rename_map)
```

- We shall ensure data types.
- Drop/impute missing values.
- Add: `date`, `year`, `month` for analysis.

```
1 data["rating"] = pd.to_numeric(
2     data["rating"],
3     errors="coerce")
4
5 data["timestamp"] = pd.to_numeric(
6     data["timestamp"],
7     errors="coerce"
8 )
9
10 # UNIX timestamps -> date time
11 data["date"] = pd.to_datetime(
12     data["timestamp"],
13     unit="s",
14     errors="coerce"
15 )
16 data["year"] = data["date"].dt.year
17 data["month"] = data["date"].dt\
18     .to_period("M").astype(str)
19
20 # Drop NA
21 data = data.dropna()
22
23 print(data.shape)
```

(1348246, 9)

Overall: KPIs

- Four **KPIs** are
 - Toal reviews
 - Number of customers
 - Number of products
 - Average rating

► Code

KPI	
tot_rev	1.348246e+06
num_cus	8.837530e+05
num_prod	2.383800e+04
avg_rating	4.145732e+00

- We build a function `make_kpi()`.

```
1 import plotly.graph_objects as go
2
3 def make_kpi(value, title, color, value_format=",d"):
4     fig = go.Figure() # initalize `go` object
5
6     fig.add_trace(      # add trace as `indicator`
7         go.Indicator(
8             mode="number",
9             value=value, # `value` argument is used.
10            title={
11                "text": title,
12                "font": {
13                    "size": 15,
14                    "color": color
15                }
16            },
17            number={
18                "valueformat": value_format,
19                "font": {
20                    "size": 28,
21                    "color": "#F5F5F5" # white text
22                }
23            }
24        )
25    )
26    fig.update_layout(
27        paper_bgcolor="#454545", # bk color
28        plot_bgcolor="#454545",  # bk color
29        margin=dict(l=10, r=10, t=90, b=10),
30        height=140
31    )
32
33    return fig
```

Overall: KPIs (Cont.)

- Apply on Total reviews:

► Code

Total Reviews
1,348,246

- On Total customers:

Unique Customers
883,753

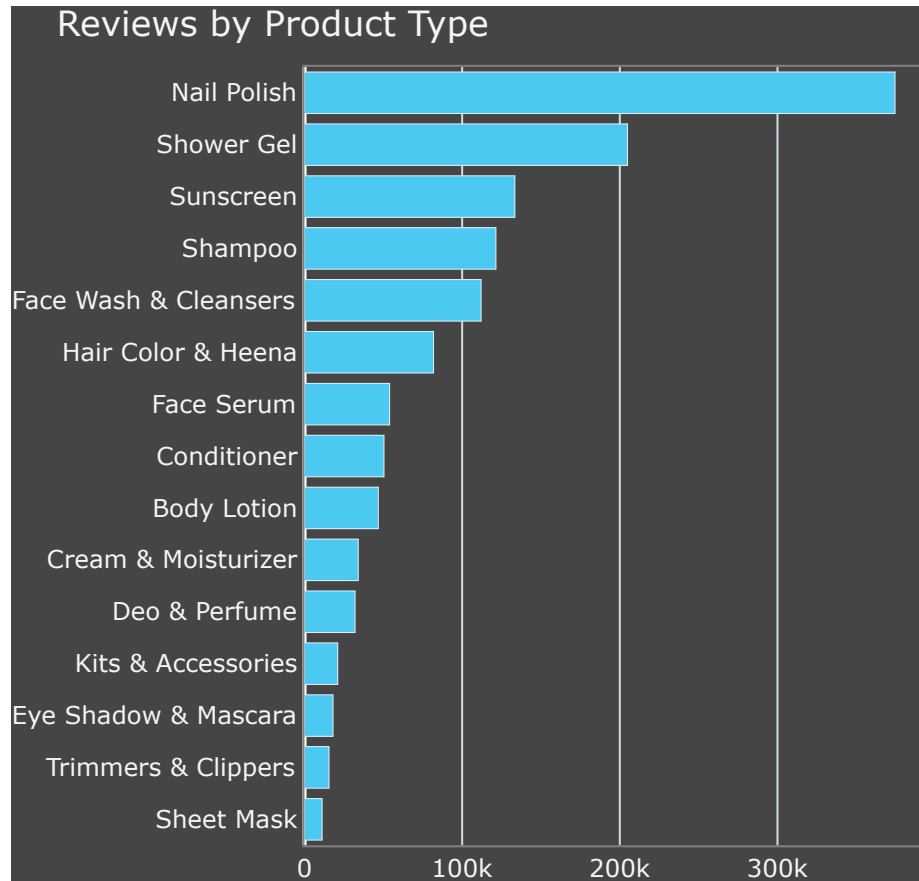
- We build a function `make_kpi()`.

```
1 import plotly.graph_objects as go
2
3 def make_kpi(value, title, color, value_format=",d"):
4     fig = go.Figure() # initialize `go` object
5
6     fig.add_trace(      # add trace as `indicator`
7         go.Indicator(
8             mode="number",
9             value=value, # `value` argument is used.
10            title={
11                "text": title,
12                "font": {
13                    "size": 15,
14                    "color": color
15                }
16            },
17            number={
18                "valueformat": value_format,
19                "font": {
20                    "size": 28,
21                    "color": "#F5F5F5" # white text
22                }
23            }
24        )
25    )
26    fig.update_layout(
27        paper_bgcolor="#454545", # bk color
28        plot_bgcolor="#454545",  # bk color
29        margin=dict(l=10, r=10, t=90, b=10),
30        height=140
31    )
32
33    return fig
```

Graphs

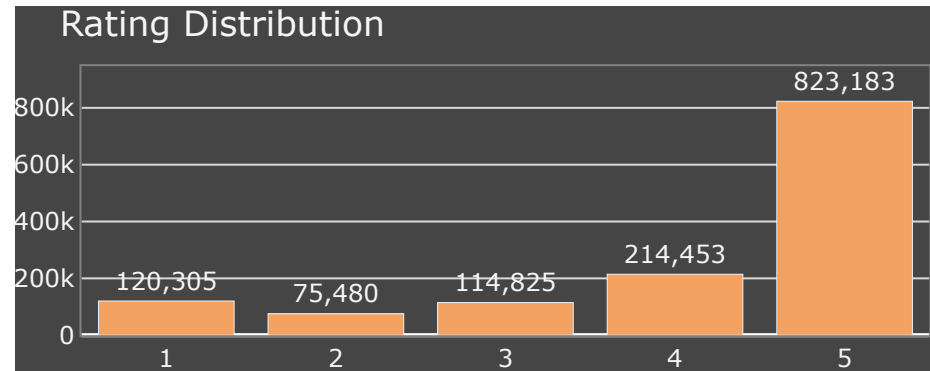
- Reveiws by product types.

► Code

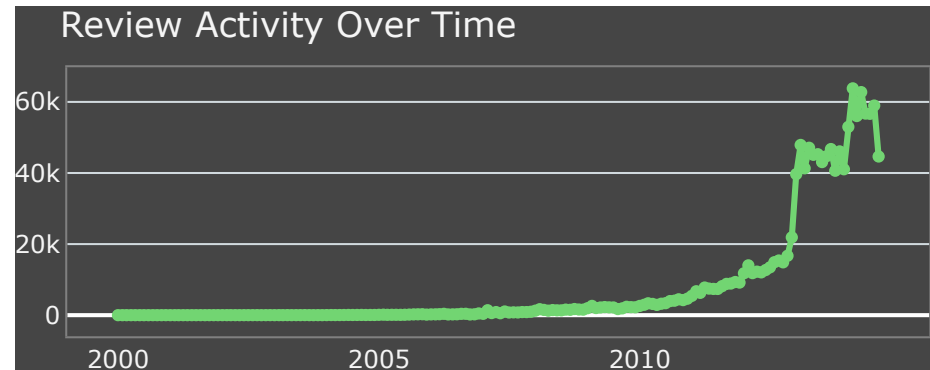


- Rating & Reveiw Trend:

► Code



► Code



Assemble Everything

- We assemble everything using `make_subplots()`:

```
1 # Assemble everything with make_subplots
2 from plotly.subplots import make_subplots
3 fig_overall = make_subplots(
4     rows=2,      # number of rows
5     cols=12,     # number of cols but break into 4 and 3
6     specs=[
7         [
8             {"type": "indicator", "colspan": 3}, None, None,
9             {"type": "indicator", "colspan": 3}, None, None,
10            {"type": "indicator", "colspan": 3}, None, None,
11            {"type": "indicator", "colspan": 3}, None, None
12        ],
13        [
14            {"type": "bar", "colspan": 4}, None, None, None,
15            {"type": "bar", "colspan": 4}, None, None, None,
16            {"type": "scatter", "colspan": 4}, None, None, None
17        ]
18    ], # Define different cols between 1st and 2nd row.
19    subplot_titles=[
20        "Total Reviews",
21        "Total Customers",
22        "Total Products",
23        "Average Rating",
24        "Reviews by Product Type",
25        "Rating Distribution",
26        "Review Activity Over Time"
27    ], # Define subplots' names
28    vertical_spacing=0.05,
29    horizontal_spacing=0.05
30 )
```

- Think about it as an empty canvas to be painted on.

- KPI can be added as `trace` to the empty canvas.

```
1 # Add KPI traces
2 fig_overall.add_trace(
3     kpi_reviews.data[0],
4     row=1, col=1
5 )
6 fig_overall.add_trace(
7     kpi_customers.data[0],
8     row=1, col=4
9 )
10 fig_overall.add_trace(
11     kpi_products.data[0],
12     row=1, col=7
13 )
14 fig_overall.add_trace(
15     kpi_rating.data[0],
16     row=1, col=10
17 )
```

- The rests and their properties are added the same way:

► Code

Assemble Everything (Cont.)

- Put the boxes around KPIs:
- Final full layout touch.

```
1 # Define the four KPI card positions
2 kpi_boxes = [
3     (0.010, 0.215, KPI_COLORS[0]),
4     (0.275, 0.470, KPI_COLORS[1]),
5     (0.535, 0.730, KPI_COLORS[2]),
6     (0.795, 1.000, KPI_COLORS[3])
7 ]
8
9 for x0, x1, color in kpi_boxes:
10     fig_overall.add_shape(
11         type="rect",
12         xref="paper",
13         yref="paper",
14         x0=x0,
15         x1=x1,
16         y0=0.58,
17         y1=0.98,
18
19         fillcolor=CARD_COLOR,
20         line=dict(
21             color=color,
22             width=2
23         ),
24         layer="below"
25     )
```

```
1 # Full layout
2 fig_overall = fig_overall.update_layout(
3     title=dict(
4         text="Amazon Product Reviews – Overall View",
5         font=dict(
6             size=24,
7             color=TEXT_COLOR
8         ),
9         x=0.02,
10        xanchor="left"
11    ),
12    height=530,
13    width=970,
14    paper_bgcolor=BG_COLOR,
15    plot_bgcolor=BG_COLOR,
16    showlegend=False,
17    margin=dict(
18        l=60, r=40, t=90, b=50
19    ),
20    font=dict(
21        family="Arial",
22        color=TEXT_COLOR
23    )
24 )
```

- **Process:** init. `go` objects → `make_subplots()`.
- One can also do: init. `make_subplots` ← `go`.

Final Result

```
1 fig_overall.show()
```

Amazon Product Reviews — Overall View

Total Reviews

Total Reviews
1,348,246

Total Customers

Total Customers
883,753

Total Products

Total Products
23,838

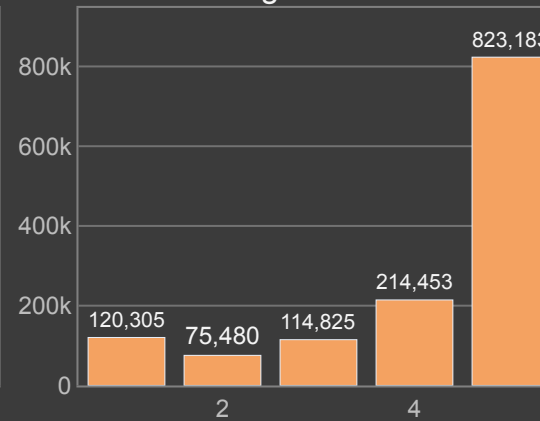
Average Rating

Average Rating
4.15

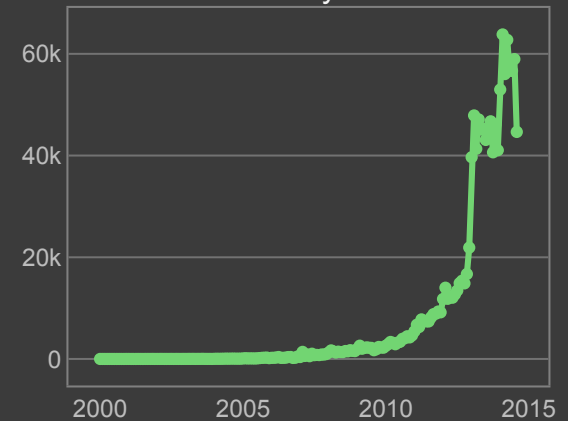
Reviews by Product Type



Rating Distribution



Review Activity Over Time



3 Customer View

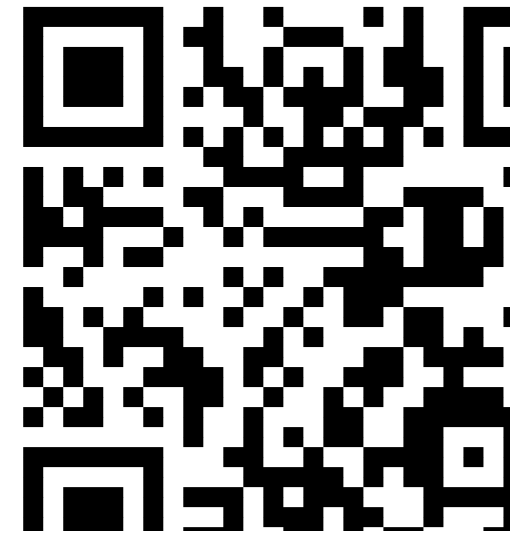
3.1 Customer View

- What aspects of customers would you consider given the following information:

`['user_id', 'product_id', 'product_type', 'rating', 'date', 'year', 'month']`

slido

Join at
slido.com
#2969 428



3.1 Customer View

- Customer information we can compute:
 - How engaged are they?
 - What do they buy / review?
 - How do their ratings behave?
 - How active they are?...
- We compute those aspects:

```
1 customer_summary = (  
2     data.groupby("user_id")  
3     .agg(  
4         reviews=("rating", "size"),  
5         products=("product_id", "nunique"),  
6         product_types=("product_type", "nunique"),  
7         avg_rating=("rating", "mean"),  
8         first_review=("date", "min"),  
9         last_review=("date", "max")  
10    )  
11    .reset_index()  
12 )
```

```
1 customer_summary["active_days"] = (  
2     customer_summary["last_review"]  
3     - customer_summary["first_review"]  
4 ).dt.days  
5  
6 customer_summary["is_repeat"] = (  
7     customer_summary["reviews"] > 1  
8 )
```

- This's the core table for customer view.

Customer KPIs

- Number of customers.
- Average number of reviews.
- Average rating per customer.
- % of repeat customers.

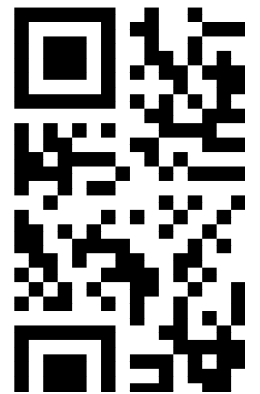
```
1 total_customers = (  
2     customer_summary["user_id"].nunique()  
3 )  
4 avg_reviews_customer = (  
5     customer_summary["reviews"].mean()  
6 )  
7 repeat_customer_rate = (  
8     customer_summary["is_repeat"].mean() * 100  
9 )  
10 avg_customer_rating = (  
11     customer_summary["avg_rating"].mean()  
12 )
```

- These are four indicators for our **customer dashboard**.
- We start from **make_subplots** and eventually add more **elements**.

- What are those **elements**?
 1. Highly engaged customers.
 2. Highly satisfied customers.
 3. Satisfaction vs engagement.
- What graph would you use for each?

slido

Join at
slido.com
#2969 428



Customer KPIs

- Number of customers.
- Average number of reviews.
- Average rating per customer.
- % of repeat customers.

```
1 total_customers = (  
2     customer_summary["user_id"].nunique()  
3 )  
4 avg_reviews_customer = (  
5     customer_summary["reviews"].mean()  
6 )  
7 repeat_customer_rate = (  
8     customer_summary["is_repeat"].mean() * 100  
9 )  
10 avg_customer_rating = (  
11     customer_summary["avg_rating"].mean()  
12 )
```

- These are four indicators for our customer dashboard.
- We start from `make_subplots` and eventually add more elements.

- Initialize `fig_customer` of `make_subplots` object.

```
1 fig_customer = make_subplots(  
2     rows=2,  
3     cols=12,  
4     specs=[  
5         # Row 1: 4 KPI  
6         [  
7             {"type": "indicator", "colspan": 3}, None, None,  
8             {"type": "indicator", "colspan": 3}, None, None,  
9             {"type": "indicator", "colspan": 3}, None, None,  
10            {"type": "indicator", "colspan": 3}, None, None  
11        ],  
12        # Row 2: 2 charts  
13        [  
14            {"type": "bar", "colspan": 4}, None, None, None,  
15            {"type": "xy", "colspan": 4}, None, None, None,  
16            {"type": "bar", "colspan": 4}, None, None, None  
17        ],  
18    ],  
19    subplot_titles=[  
20        "Total Customers", "Avg Review", "Repeat Rate", "Avg Rating",  
21        "Customer Reviews",  
22        "Customer Engagement vs Satisfaction",  
23        "Customers Satisfaction"  
24    ],  
25    vertical_spacing=0.05,  
26    horizontal_spacing=0.08  
27 )
```

Customer KPIs (Cont.)

- We then add all the KPIs:

```
1 # Add number of customers
2 fig_customer = fig_customer.add_trace(
3     go.Indicator(
4         mode="number",
5         value=total_customers, # alr computed
6         title=dict(
7             text="Total Customers",
8             font=dict(
9                 size=15,
10                color=KPI_COLORS[0]
11            )
12        ),
13        number=dict(
14            valueformat=",d",
15            font=dict(
16                size=28,
17                color=TEXT_COLOR
18            )
19        )
20    ),
21    row=1,
22    col=1
23 )
```

- The rests are added the same way.

► Code

- We add boxes to KPIs:

```
1 KPI_BOXES = [
2     (0.01, 0.185),
3     (0.245, 0.485),
4     (0.520, 0.750),
5     (0.815, 0.998)
6 ]
7 for (x0, x1), color in zip(
8     KPI_BOXES,
9     KPI_COLORS):
10     fig_customer = fig_customer.add_shape(
11         type="rect",
12         xref="paper",
13         yref="paper",
14         x0=x0,
15         x1=x1,
16         y0=0.58,
17         y1=0.98,
18         fillcolor=CARD_COLOR,
19         line=dict(
20             color=color,
21             width=2),
22         layer="below")
```

- Current result can be viewed using `fig_customer.show()`.

Customer Charts

1. Highly engaged customers.

```
1 # Sort reviews
2 high_engagement = customer_summary\
3     .nlargest(10, "reviews")\
4     .sort_values('reviews',
5                 ascending=False)
6 fig_customer = fig_customer.add_trace(
7     go.Bar(
8         y=high_engagement["reviews"],
9         x=high_engagement["user_id"].astype(str),
10        orientation="v",
11        marker=dict(
12            color=KPI_COLORS[0]
13        ),
14        hovertemplate=(
15            "<b>Customer: %{x}</b><br>"
16            "Reviews: %{y:,}"
17            "<extra></extra>"
18        )
19    ),
20    row=2,
21    col=1
22 )
```

- Highly engagement can indicate potential customers or problems (fake reviews).

2. Highly satisfied customers.

```
1 # Sort reviews
2 satisfied_customers = (
3     data.assign(
4         high_rating=data['rating'].isin([4,5])
5     ).groupby("user_id")
6     .agg(
7         high_ratings=("high_rating", 'sum'),
8         reviews=("rating", "size"),
9         avg_rating=("rating", "mean")
10    )
11    .reset_index()
12    .nlargest(10, "high_ratings")
13    .sort_values("high_ratings", ascending=False)
14 )
15 fig_customer = fig_customer.add_trace(
16     go.Bar(
17         y=satisfied_customers["high_ratings"],
18         x=satisfied_customers["user_id"].astype(str),
19         orientation="v",
20         marker=dict(
21             color=KPI_COLORS[3]),
22         hovertemplate=(
23             "<b>Customer: %{x}</b><br>"
24             "High Rating Count: %{y:,}"
25             "<extra></extra>"
26         )
27     ),
28     row=2,
29     col=9
30 )
```

Customer Charts (Cont.)

- Engagement vs satisfaction.

```
1 comparison = (  
2     customer_summary[  
3         ["user_id", "reviews", "avg_rating"]  
4     ]  
5     .merge(  
6         satisfied_customers[  
7             ["user_id", "high_ratings"]  
8         ],  
9         on="user_id",  
10        how="left"  
11    )  
12 )  
13  
14 comparison["high_ratings"] = (  
15     comparison["high_ratings"]  
16     .fillna(0)  
17 )
```

- **Scatter plot** is a good graph:

► Code

Final Touch



► Code

Amazon Product Reviews — Customer View

Total Customers

Total Customers

883,753

Avg Review

Avg Reviews / Customer

1.5

Repeat Rate

Repeat Customer Rate

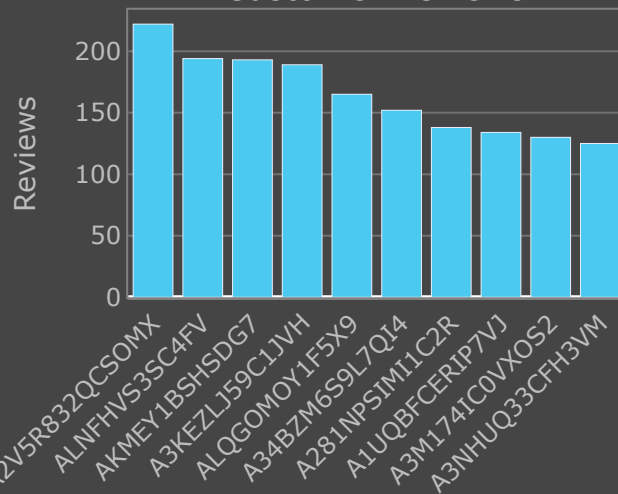
22.9%

Avg Rating

Avg Rating

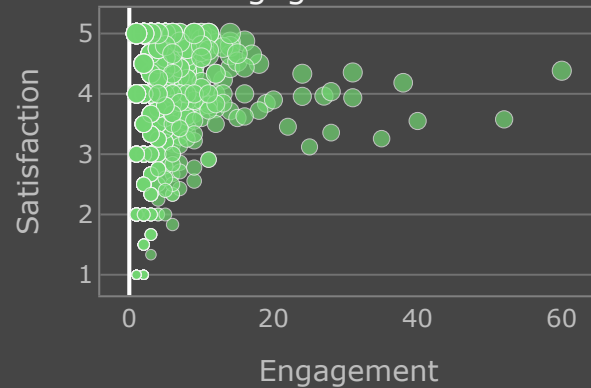
4.12

Customer Reviews



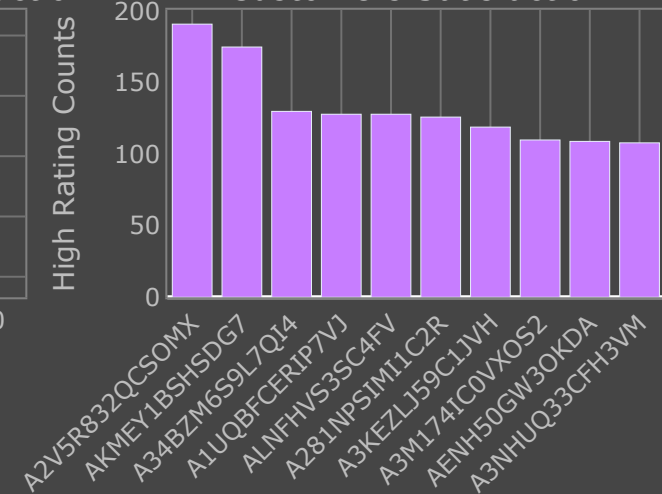
Customers

Customer Engagement vs Satisfaction



Engagement

Customers Satisfaction



Customers

4 Product View

4.1 Product Information

- Based on the same principle, we set the following KPIs:
 - Total products
 - Percentage of High Rated Products
 - Products Needing Attention
- Charts information:
 - Top Rated Products
 - Lowest Rated Products
 - Product Rating vs. Review Volume

► Code

Product Dashboard

► Code

Amazon Product Reviews — Product View

Total Product

Total Product
23,838

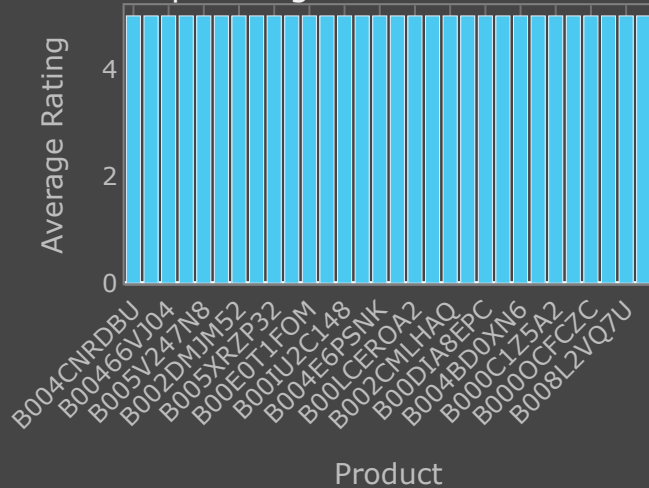
High Rated Products

High Rated Products
70.7%

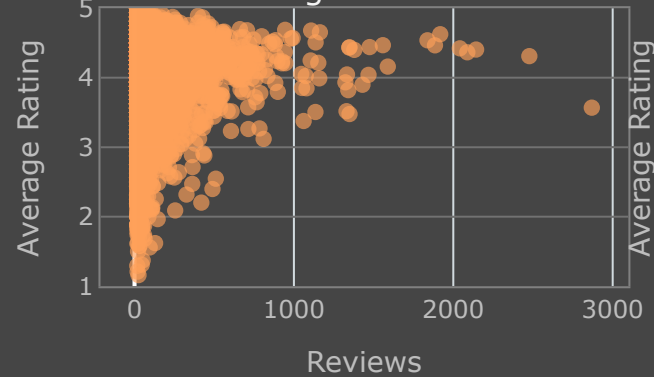
Products Needing Attention

Products Needing Attention
1.3%

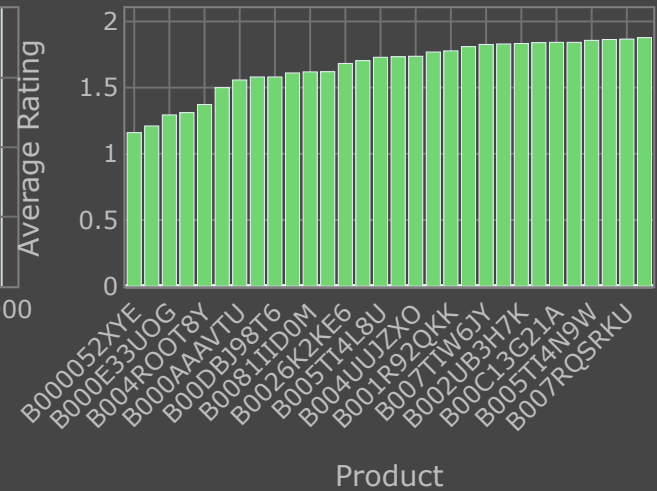
Top 30 Highest Rated Products



Product Rating vs. Review Volume



Bottom 30 Lowest Rated Products



5 Trending View

5.1 Trending View

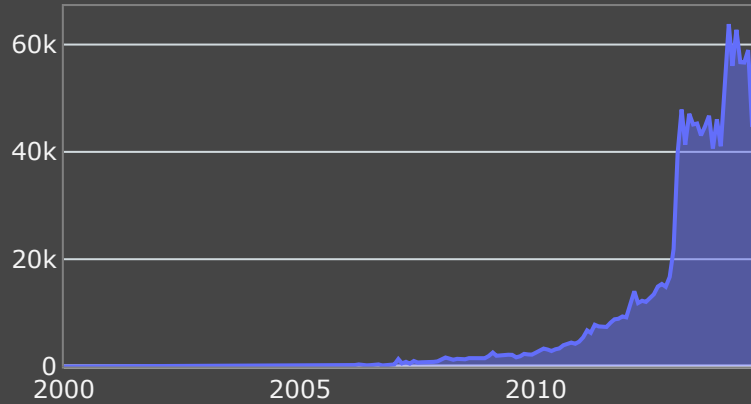
- It's important to highlight the trend of various indicators:
 - Number of reviews (engagement)
 - Average Rating
 - Distribution of Rating
 - Rating evolution of some popular product types...
 - **Lineplot** would be the main graph for this purpose.
 - We first perform some data manipulation:
- Code

Trending Dashboard

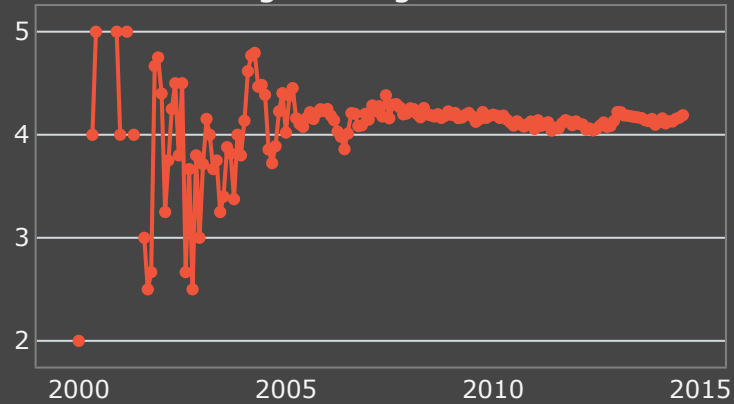
► Code

Amazon Beauty Products — Rating & Market Trends

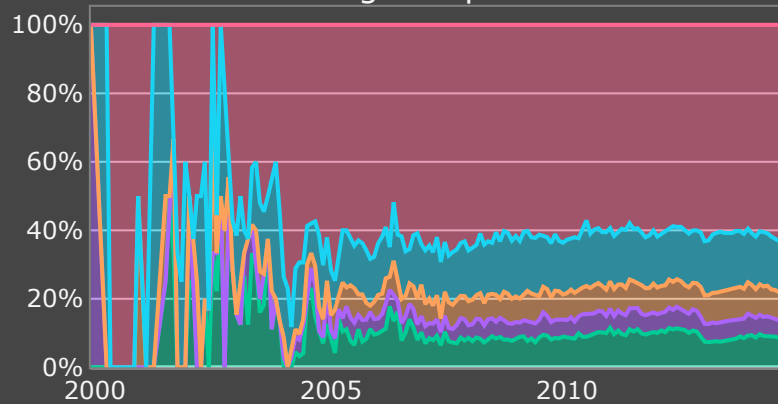
Review Volume Over Time



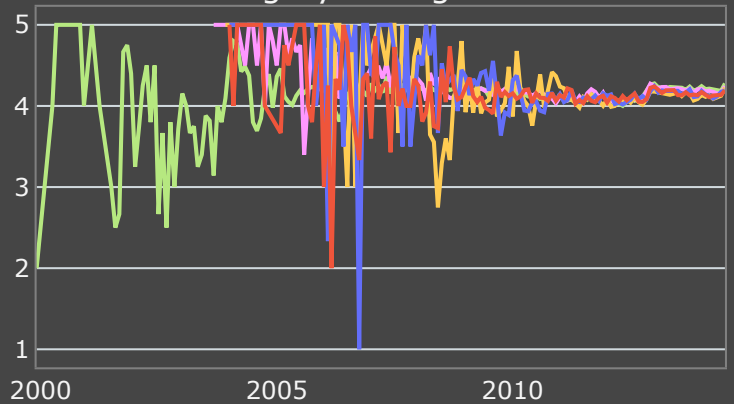
Average Rating Over Time



Rating Composition



Category Rating Trends



- Face Wash & Cleansers
- Shampoo
- Sunscreen
- Shower Gel
- Nail Polish
- 5★
- 4★
- 3★
- 2★
- 1★
- Avg Rating evolution
- Review evolution

6 Final Dashboard

6.1 Quarto Tabs

Overview

Customer

Product

Trend

Amazon Product Reviews — Overall View

Total Reviews

Total Reviews
1,348,246

Total Customers

Total Customers
883,753

Total Products

Total Products
23,838

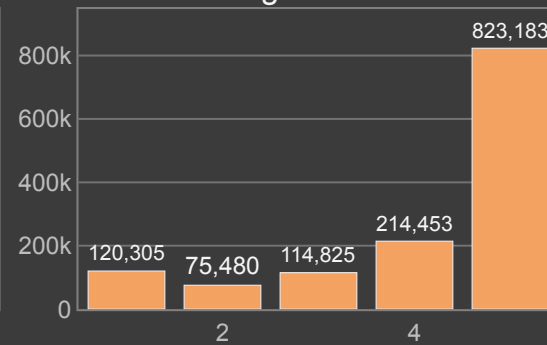
Average Rating

Average Rating
4.15

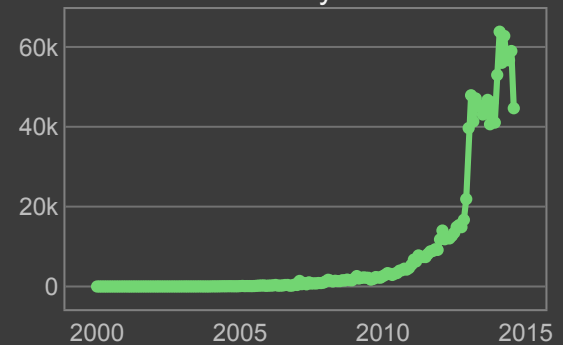
Reviews by Product Type



Rating Distribution



Review Activity Over Time



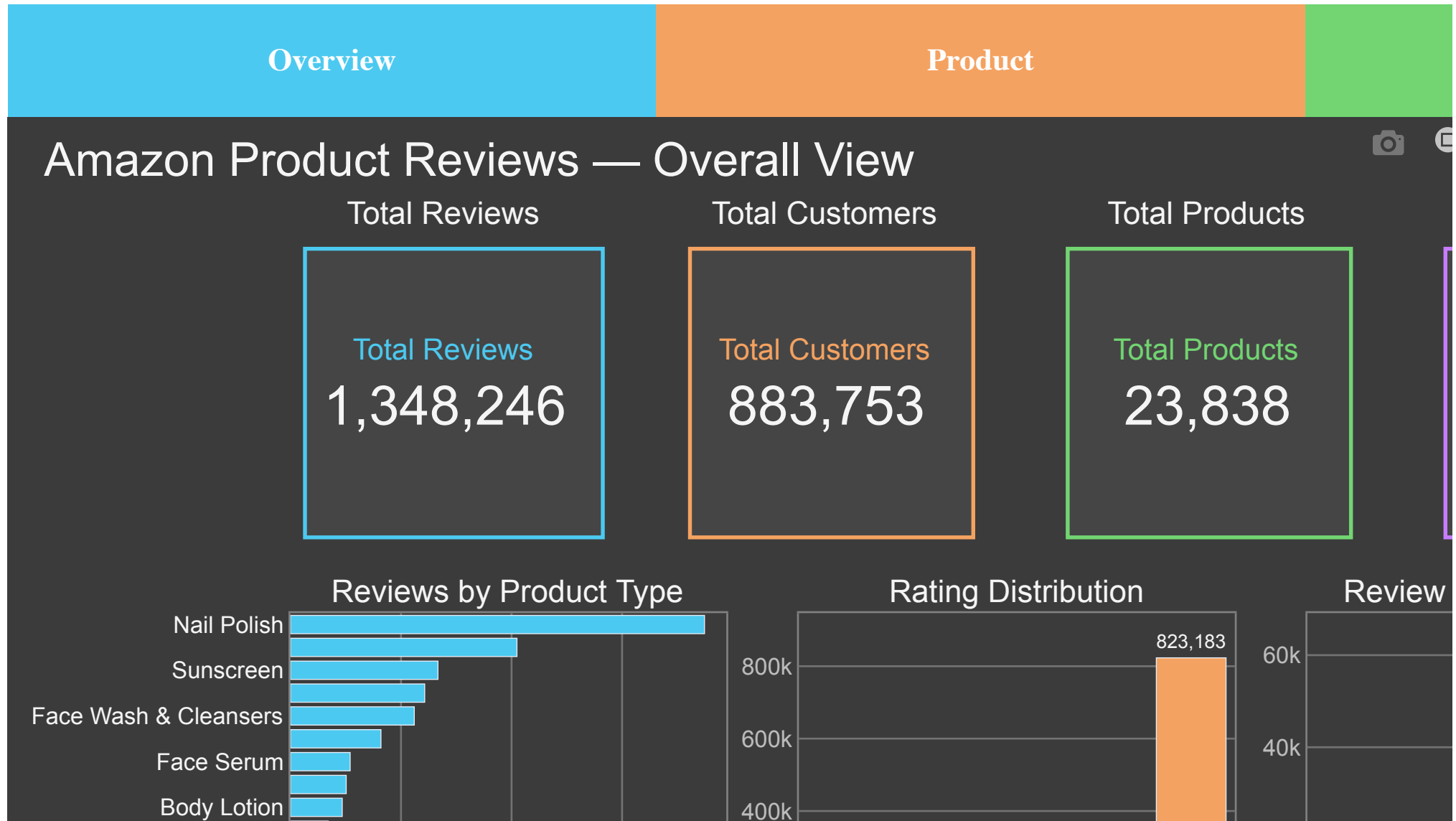
6.2 Dash Framework

- Dumb all the figures then assemble them back using **dash**.

```
1 import pickle
2
3 figures = {
4     "overview": fig_overall,
5     "product": fig_product,
6     "customer": fig_customer,
7     "trend": fig_trend
8 }
9
10 with open("dashboard_figures.pkl", "wb") as f:
11     pickle.dump(figures, f)
```

- **Dash** is a great tool for dashboard development and deployment.
- We can then read back the 4 figures and assemble them into a nice dashboard:  [Amazon dashboard](#).

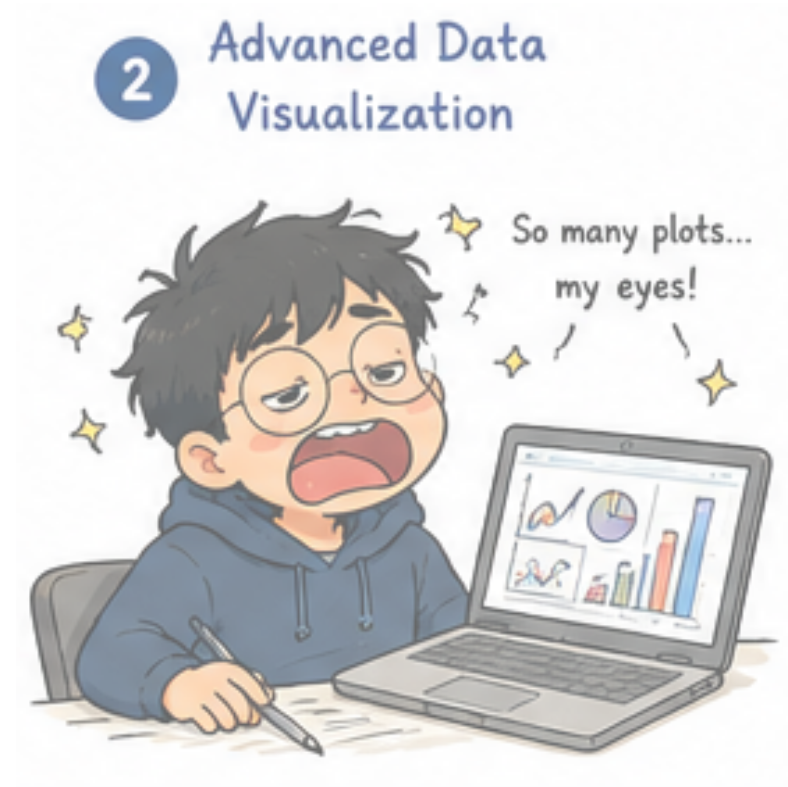
6.2 Dash Framework



6.3 Summary

- **Business objective** is always the main driver of how the dashboard should look like.
- **Define KPIs** that highlights the indicators to the main objective.
- **Data Wrangling and Manipulation** is another important step to start building the dashboard.
- **Select format and aesthetic** of the dashboard.
- **Python & Plotly** are tools for CS students.
- **Power BI and Tableau** are for business and less technical people.
- **AI Tools such as Claude, Gemini...** are also great but you have to know what you are doing.

🥳 Yeahhhh 🍷 !!!



Any question? Take a break!